

AIKernel Hash-Anchored Trust Layer (HATL): ハッシュベース公開アンカーを備えたハイブリッド対称レジャー

確率的 AI 推論トラックを保護する対称ラチェット型マイクロレジャー

Takuya Sogawa

2026-06-02

AIKernel Hash-Anchored Trust Layer (HATL): ハッシュベース公開アンカーを備えたハイブリッド対称レジャー

著者: Takuya Sogawa

所属: AIKernel Project

ORCID: <https://orcid.org/0009-0009-7499-2595>

DOI: <https://doi.org/10.5281/zenodo.20502685>

Version: v1.0

文書ライセンス: CC-BY-4.0

コード仕様ライセンス: Apache-2.0

本文書は Part I の英語正本に対する技術参考訳である。解釈に差異がある場合は、Part I: Full English Manuscript を正とする。

概要

自律型 AI システムは、高頻度の推論呼び出し、状態遷移、検索処理、ツール呼び出し、ケイパビリティ更新を連続的に実行する。このようなシステムにおける実行記録は、単なるアプリケーションログではない。再現性、説明責任、インシデント分析、ロールバック判断の根拠となる監査基盤である。したがって、AIKernel のような Semantic Context Operating System には、推論トラックと ReplayLog を決定論的な完全性をもって保持し、かつランタイムで利用可能な速度を維持できるトラストレイヤーが必要となる。

本稿では、**AIKernel Hash-Anchored Trust Layer (HATL)** を提案する。HATL は、高頻度のローカルコミットと、低頻度の公開アンカーリングを分離するハイブリッド型トラストアーキテクチャである。内部レイヤーは、HMAC-SHA-512、HKDF、ハッシュチェーン、Merkle 集約に基づく対称ラチェット型マイクロレジャーである。外部レイヤーは、一定間隔で状態ルートを公開アンカーへ固定する。厳密な状態管理が可能な環境では LMS/XMSS などの stateful hash-based signatures を、運用単純性を優先する環境では stateless な SLH-DSA を用いる。

HATL は、すべての推論ステップに重い公開鍵署名を直接付与することを避ける。代わりに、各ランタイムイベントを前方ラチェットされたローカルレジャーへコミットし、ポリシーで定められた間隔でバッチ化されたルートを公開する。これにより、ランタイムは低レイテンシのローカル完全性を得つつ、監査者はコンパクトな公開検証経路を得ることができる。

本稿の貢献は、新しい暗号プリミティブの提案ではなく、既存の標準プリミティブを AIKernel の実行証跡保護に適した階層構造へ合成する点にある。HATL の安全性主張は、基礎プリミティブ、鍵ライフサイクル管理、正準化、公開台帳の最終性ポリシーが正しく実装されることを前提とする。

キーワード

AIKernel, HATL, ハッシュベース署名, 対称レジャー, HMAC, HKDF, Merkle Tree, ReplayLog, Digital Deed, Semantic Context Operating System, AI Governance, Forward Secrecy, Public Anchoring, Post-Quantum Transition.

1. 序論

LLM を中心とするソフトウェアシステムは、単なるテキスト生成器から、自律的または半自律的な実行主体へと進化している。これらはコンテキストを読み込み、構造化された計画を生成し、ツールを呼び出し、タスク状態を更新し、外部システムと連携す

る。AIKernel の用語では、この一連の振る舞いは推論トラックを生成する。推論トラックとは、入力コンテキスト、中間判断、出力、ツール呼び出し、ケイパビリティ検査、実行結果の系列である。

Semantic Context Operating System において、この推論トラックは再現可能かつ監査可能でなければならない。あるツール呼び出しが許可されたならば、後から、どのコンテキストが存在し、どのケイパビリティが有効で、どのポリシーが適用され、どの遷移がコミットされたのかを証明できなければならない。

単純な方法は、すべてのイベントに公開鍵署名を付けることである。しかし高頻度の推論システムでは、1 秒あたり多数の小さなイベントが発生する。すべての内部ステップに重い公開鍵署名、特に耐量子署名を適用すると、レイテンシとストレージの負担が大きくなる。一方、対称鍵認証だけでは、第三者が秘密鍵なしに検証できないため、公開監査性に欠ける。

HATL はこの緊張関係を、信頼ドメインの分離によって解消する。

1. **内部対称ドメイン:** 高頻度のローカルコミットを HMAC と HKDF ラチェットで処理する。
2. **集約ドメイン:** ローカルブロックを Merkle root へ集約する。
3. **公開アンカードメイン:** Merkle root をハッシュベース公開署名で定期的に固定する。
4. **ガバナンスドメイン:** Digital Deed の状態、失効、ポリシーエポックを管理する。

中心原理は単純である。ランタイム完全性と公開監査性は、同じ頻度で実行される必要はない。HATL は、ランタイムイベントを対称暗号の速度でコミットし、公開検証をコンパクトな状態ルートに対して行う。

1.1 貢献

本稿の主な貢献は以下である。

- **ハイブリッド型トラストレイヤー:** 内部対称マイクロレジャーと外部ハッシュベース公開アンカーを組み合わせた HATL を定義する。
- **ラチェット型対称マイクロレジャー:** 各イベントを前回コミットメントに結合し、アクティブ鍵を HKDF により前方進化させる BlockMAC 構造を定義する。
- **ハッシュベース公開アンカー:** LMS/XMSS または SLH-DSA によって Merkle root を公開固定する間隔ベースのモデルを定義する。
- **Digital Deed ガバナンス:** ノードのアイデンティティ、ケイパビリティ、失効状態を管理する Digital Deed とクォーラム失効意味論を導入する。
- **実装規律:** 再現可能なポジトリ構成、インターフェース案、スキーマ、サイドチャネルを考慮した実装規律を提示する。
- **非主張事項の明示:** 形式証明、ポスト量子保証、性能値を過剰に主張しないため、主張範囲を明確化する。

1.2 範囲と非主張事項

HATL は暗号プリミティブそのものではなく、それらを組み合わせるアーキテクチャである。本稿は以下を主張しない。

1. HATL が既存の認証済みデータ構造の形式解析を置き換えること。
2. 任意のブロックチェーンがあらゆる条件下で絶対的最終性を提供すること。
3. HMAC、HKDF、LMS、XMSS、SLH-DSA を本稿が新規に発明したこと。
4. Stateful HBS が厳密な状態管理なしに安全に利用できること。
5. 試作スループット値が普遍的なベンチマークであること。
6. 公開台帳にハッシュを固定するだけで全ての法令遵守要件を満たすこと。

本稿の主張は、より限定的である。すなわち、HATL は高速なローカル対称コミットメントと、より低頻度だが公開検証可能なハッシュベースアンカーを結合する実用的な設計パターンを提示する。

2. 背景と関連技術

2.1 HMAC と HKDF

HMAC は暗号学的ハッシュ関数に基づく鍵付きメッセージ認証構成である。HATL では HMAC-SHA-512 を用いてローカルレジャーブロックを認証する。HKDF は、過去の鍵材料と実行コンテキストから前方進化するエポック鍵を導出するために用いる。

2.2 ハッシュベース署名

ハッシュベース署名は、離散対数や格子問題のような代数的困難性ではなく、主としてハッシュ関数の性質に依存するため、長期的なトラストアンカーとして魅力的である。HATL では以下を区別する。

- **Stateful HBS:** LMS と XMSS は保守的で効率的だが、ワンタイム署名の再利用を防ぐ厳密な状態管理が必要である。

- **Stateless HBS:** SLH-DSA は永続的な署名状態を持たないため運用が単純だが、署名サイズや性能に別のトレードオフがある。

2.3 Merkle 集約

Merkle Tree は、イベントコミットメント列を単一の root へ圧縮する。HATL は、すべてのイベントを公開台帳へ書き込むのではなく、アンカーウィンドウごとの root のみを署名・公開する。

2.4 Semantic Context Operating System

AIKernel は、AI 実行を OS 的に扱う。コンテキストは統治されたメモリ、ツールはケイパビリティ、実行はポリシーによって媒介され、ReplayLog は監査オブジェクトとなる。HATL は、この実行トラックを改ざん検知可能かつ外部アンカー可能にする暗号的トラストレイヤーである。

3. 脅威モデルとセキュリティ目標

3.1 脅威モデル

本稿では以下の能力を持つ攻撃者を想定する。

- **リモート入力操作:** プロンプトインジェクション、破損 material の挿入、ケイパビリティ昇格の試み。
- **ストレージ抽出:** ローカルデータベース、ROM スナップショット、キャッシュ、ReplayLog アーカイブの漏洩。
- **実行時鍵露出:** 現在の対称エポック鍵が一時的に露出する状況。
- **台帳層干渉:** 最終性遅延、チェーン再編成、アンカー検閲、無効アンカー再送。
- **ガバナンス操作:** 偽の失効提案、不十分なクォーラム署名の提出。

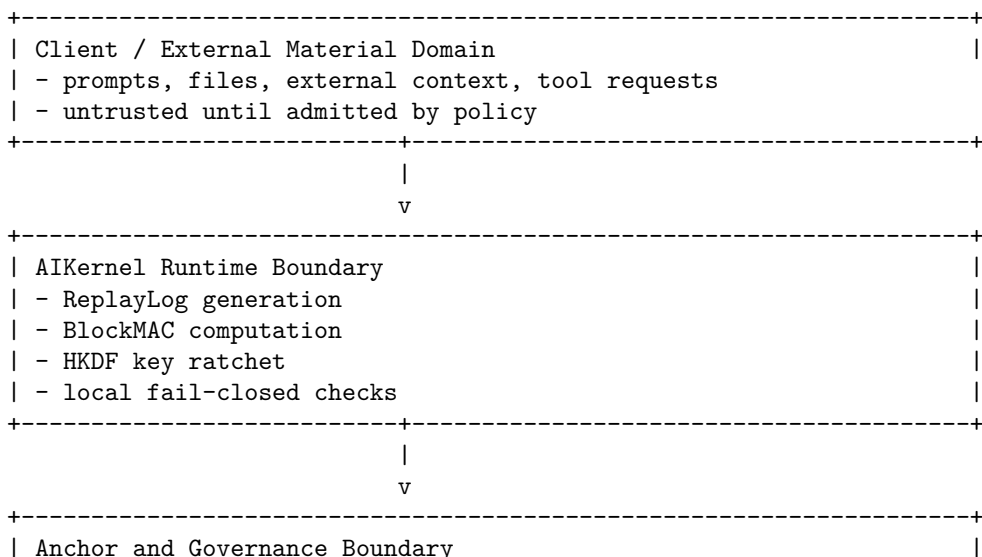
将来の量子計算能力も考慮するが、無条件安全性は主張しない。安全性は、選択したハッシュ関数、鍵管理、実装、アンカーポリシーに依存する。

3.2 セキュリティ目標

HATL は以下を目標とする。

- **ローカル鍵の前方秘匿性:** 現在鍵の漏洩が過去鍵の導出につながらないこと。
- **改ざん検知性:** コミット済みイベントの変更が、ローカルチェーン、Merkle root、公開アンカーのいずれかで検知されること。
- **限定された公開監査性:** 外部監査者が、対称鍵を受け取ることなく、イベントが公開アンカーウィンドウに含まれることを検証できること。
- **失効執行性:** 侵害された Digital Deed を有効実行集合から除外できること。
- **Fail-Closed:** 信頼状態が曖昧または無効な場合は、特権実行を停止すること。

3.3 信頼境界



- Merkle root publication	
- HBS signature verification	
- Digital Deed registry	
- quorum-based revocation	
+-----+	

4. HATL アーキテクチャ

4.1 ランタイムフロー

HATL は推論実行トラックを次のような暗号コミットメント列へ写像する。

```
Inference Event
-> canonical event representation
-> event hash
-> BlockMAC using current epoch key
-> HKDF ratchet to next epoch key
-> Merkle leaf accumulation
-> periodic root anchoring
-> public anchor verification
```

内部ループは対称暗号で高速に実行され、外部ループは公開検証可能だが低頻度に行われる。この分離が HATL の中心的な性能・ガバナンス上の考え方である。

4.2 BlockMAC レコード

BlockMAC はイベント、直前の因果的先行ブロック、現在のポリシーコンテキストへコミットする。

```
{
  "sequence_number": 1024,
  "epoch_id": 7,
  "previous_block_mac": "hex-encoded-64-byte-value",
  "event_hash": "hex-encoded-64-byte-value",
  "execution_outcome_hash": "hex-encoded-64-byte-value",
  "capability_scope_hash": "hex-encoded-64-byte-value",
  "hmac_signature": "hex-encoded-64-byte-value"
}
```

HMAC が認証する対象は、`hmac_signature` を除く全フィールドの決定論的正準シリアライズである。JCS などの決定論的形式を用いることで、プラットフォーム差を排除する。

4.3 鍵ラチェット

時刻 t の有効エポック鍵を K_t 、イベントまたは実行結果のハッシュを E_t とする。次の鍵は以下で導出される。

$$K_{t+1} = HKDF_Expand(HKDF_Extract(salt_t, K_t), info = E_t \parallel t, L = 64)$$

有効な実装では、 K_{t+1} の導出および BlockMAC 計算後、 K_t は消去またはアクセス不能化されなければならない。過去鍵が保持される場合、前方秘匿性は弱まる。特に C#/NET のようなマネージドランタイムでは、通常のガベージコレクションが秘密鍵の即時消去を保証するわけではないため、`Span<byte>` / `ReadOnlySpan<byte>`、必要に応じた pinned memory または native memory、ならびに `CryptographicOperations.ZeroMemory` による明示的の消去を含むメモリ保護規律が必須となる。

4.4 Merkle アンカーウィンドウ

アンカーウィンドウが BlockMAC 値 $B_i \dots B_j$ を含むとする。各リーフは以下で計算される。

$$L_k = H("HATL-LEAF" \parallel canonical(B_k))$$

内部ノードは以下で計算される。

$$N_{parent} = H("HATL - NODE" \parallel N_{left} \parallel N_{right})$$

リーフと内部ノードを誤って混同しないよう、ドメイン分離ラベルを推奨する。

4.5 公開アンカー文書

```
{
  "anchor_id": 42,
  "ledger_identity": "AIKernel-HATL-demo-ledger",
  "policy_epoch": 3,
  "batch_start_seq": 1000,
  "batch_end_seq": 1999,
  "accumulated_merkle_root": "hex-encoded-root",
  "previous_anchor_hash": "hex-encoded-anchor-hash",
  "hbs_signature_type": "SLH-DSA",
  "hbs_signature_raw": "hex-or-base64-signature"
}
```

公開アンカー文書は、バッチ root と前回アンカーハッシュを結合する。これにより、公開された append-only なアンカーチェーンが構成される。実イベント本文は秘匿したまま、包含証明によって監査可能性を確保できる。

4.6 Digital Deed

Digital Deed はノードのアイデンティティとケイパビリティのアンカーである。

```
{
  "node_uuid": "3ec7fe55-9b89-4a74-9a92-cf68e9ad1a6d",
  "rom_identity_commitment": "hex-encoded-rom-commitment",
  "assigned_public_anchor_key": "encoded-public-key",
  "capability_contract_hash": "hex-encoded-capability-contract",
  "status": "ACTIVE",
  "policy_epoch": 3
}
```

Digital Deed は単なる証明書ではない。ACTIVE、SUSPENDED、REVOKED へ遷移するガバナンスオブジェクトである。

5. ガバナンスと失効

5.1 クォーラム失効

侵害されたノードは、過去のアンカーが有効であっても、将来の信頼主体として維持されてはならない。HATL は履歴検証とアイデンティティ状態を分離する。ある公開アンカーは歴史的に有効であり続ける一方で、対応する Digital Deed は後に失効し得る。

Algorithm: EvaluateRevocationProposal

Input: proposal P(target_node, evidence_hash, quorum_signatures)

Output: updated Digital Deed status or rejection

1. If P contains fewer than m valid governance signatures, reject.
2. Verify each governance signature against the configured policy epoch.
3. Verify that evidence_hash is bound to a valid audit artifact or incident record.
4. If signature validation, evidence validation, policy lookup, or network-dependent quorum resolution returns Indeterminate, reject the proposal and fail closed.
5. If verification succeeds, set DigitalDeed(target_node).status = REVOKED.
6. Broadcast the revocation event to runtime policy enforcement points.
7. Runtime nodes must fail closed for new privileged work from the revoked identity.

5.2 異議申し立てウィンドウ

運用上の安全性のため、HATL は challenge window を持ってもよい。この期間中、ノードまたは監査者は、停止または失効の根拠に対して代替監査経路を提出できる。これはガバナンスポリシーであり、暗号プリミティブではない。

5.3 ポリシーエポック

すべてのガバナンス判断はポリシーエポックに結合される。ポリシーエポックは、ルールセット、クォーラム閾値、検証者集合、アンカーアルゴリズム、正準化バージョンを識別する。

6. セキュリティ分析

6.1 前方ラチェット主張

主張: HKDF が安全なハッシュ関数で実装され、古いエポック鍵がラチェット後に消去される場合、 K_T の漏洩は $t < T$ の K_t を直接明かさない。

スケッチ: HATL は HKDF とイベント文脈により K_t から K_{t+1} を導出する。 K_{t+1} から K_t を復元するには、KDF 構成を反転するか、内部ハッシュ出力の原像に関する有用な情報を得る必要がある。HKDF と選択ハッシュ関数の標準仮定のもとで、これは計算困難である。ただし、過去鍵がメモリ、クラッシュダンプ、スワップ領域、診断ログ、またはマネージドヒープの残留領域に残っている場合、この性質は成立しない。したがって C#/.NET 配備では、秘密値の寿命を短く保ち、不要なコピーを避け、高リスクの鍵材料には pinned/native buffer を使い、バッファ解放前に `CryptographicOperations.ZeroMemory` で可能な限り消去する、という明示的な秘密管理方針が必要である。

6.2 改ざん検知主張

主張: アンカー済みイベントを変更すると、攻撃者が衝突を発見するか認証値を偽造しない限り、BlockMAC、Merkle path、または anchor root が変化する。

スケッチ: イベントは BlockMAC に寄与し、BlockMAC は Merkle leaf に寄与し、leaf は root に寄与する。root は署名または公開固定される。履歴イベントを変更すると root も変化するため、攻撃者は衝突または認証チェーンの偽造を必要とする。HATL が提供するの改ざん検知性であり、物理削除そのものを魔法のように防ぐものではない。

6.3 公開検証性主張

主張: 外部監査者は、ローカル対称鍵を受け取らずに、開示されたイベントが公開アンカーに含まれていることを検証できる。

スケッチ: 監査者は、イベントレコード、正準シリアルライズ、BlockMAC または leaf、Merkle path、公開 anchor を受け取る。監査者は root までの包含経路を再計算し、HBS 署名または台帳コミットメントを検証する。ローカル HMAC 検証は必要に応じて信頼実行境界内で行われる。

6.4 仮定と失敗モード

HATL は以下に依存する。

1. 選択したハッシュ関数および HMAC/HKDF 構成が運用期間中安全であること。
2. 正準化が決定論的かつバージョン管理されること。
3. Stateful HBS のカウンターが再利用されないこと。
4. 鍵が十分なエントロピーで生成されること。
5. 古い対称鍵が消去またはアクセス不能化されること。
6. 公開アンカーの finality policy が基盤台帳に適切であること。
7. ガバナンス鍵が保護・ローテーションされること。

7. 試作と評価計画

7.1 試作範囲

v1.0 パッケージはアーキテクチャトリポジトリ仕様を定義する。完全に認証された本番実装は含まない。Part II の参照リポジトリ仕様は、期待されるソース構成、スキーマ、コントラクト標本、テスト足場を定義する。

7.2 評価指標

指標	意味
BlockMAC throughput	1 秒あたりのローカルイベントコミット数

指標	意味
Ratchet latency	次エポック鍵導出時間
Merkle build cost	ウィンドウサイズごとの root 構築時間
Anchor publication latency	root 生成から公開コミットまでの時間
Verification latency	包含証明と公開署名の検証時間
Memory allocation profile	イベントまたはウィンドウあたりの割当バイト数
Revocation propagation time	Digital Deed 状態変化の執行時間

7.3 予備性能プロファイル

元の設計ノートには、内部対称ループについて毎秒数万コミット規模の試作目標が含まれている。本 v1.0 清書版では、これらの数値は普遍的な性能主張ではなく、**予備的な試作ターゲット**として扱う。将来版では、ベンチマークスクリプト、ハードウェア構成、コンパイラ、ランタイム設定、再現ログを同梱すべきである。

8. 配備と運用上の考察

8.1 Stateful HBS ライフサイクル

LMS または XMSS を使用する場合、状態管理は極めて重要である。クラッシュ、ロールバック、バックアップ、並行署名インスタンスを含む全状況で、ワンタイム署名リーフの再利用を防がなければならない。

8.2 SLH-DSA 代替プロファイル

SLH-DSA は署名カウンターを必要としないため、運用安全性が単純になる。その代わり、署名サイズや性能面で別のトレードオフがある。HATL は、状態管理を保証できない環境において SLH-DSA を有効なアンカープロファイルとして扱う。

8.3 プライバシーとメタデータ漏洩

アンカー root の公開は、タイミングや処理量のメタデータを明かし得る。対策として、固定サイズのアンカーウィンドウ、ポリシー範囲内のランダム遅延、null-event padding、機密環境向けの private/consortium ledger が考えられる。ただし、これらは漏洩を低減するものであり、ゼロ情報開示を証明するものではない。将来の privacy-preserving anchor profile では、zk-SNARKs などのゼロ知識証明を組み込み、マークルパス、イベント履歴、内部 ReplayLog の内容を明かさずに「当該パッチが規定のレジャー遷移規則に従って処理された」ことだけを公開検証できる構成を検討する。

8.4 法務・コンプライアンス注記

HATL が公開アンカー面に保存するのは、推論本文ではなく暗号学的コミットメントである。これはプライバシー上好ましいが、すべての法域において自動的に要件を満たすわけではない。ハッシュ、メタデータ、タイムスタンプ、リンク構造が個人データとみなされるかは個別評価が必要である。

9. リポジトリ仕様の要約

Part II には、技術付録およびリポジトリ仕様が含まれる。HATL 参照リポジトリは少なくとも以下を含むべきである。

```
/AIKernel.Hat1
  LICENSE
  README.md
  HATL_SPEC.md
/src
  /AIKernel.Hat1.Core
  /AIKernel.Hat1.Anchor
/schemas
  hatl-block-mac.schema.json
  hatl-public-anchor.schema.json
  hatl-digital-deed.schema.json
/contracts
  HatlGovernance.sol
```

```
/tests
  RatchetTests.cs
  AnchorVerificationTests.cs
  BenchmarkProfiles.json
```

コード部分は Apache-2.0、説明文書は CC-BY-4.0 とする。

10. 限界と今後の研究

HATL には以下の限界がある。

1. 既存プリミティブを合成するが、それらの形式証明を置き換えるものではない。
2. 正準化、鍵消去、HBS 状態管理の正しい実装に依存する。
3. 公開台帳の最終性は台帳ごとに異なり、確率的であり得る。
4. プライバシー対策はメタデータ漏洩を低減するが、完全に排除するとは限らない。
5. スループットはハードウェア、実装、バッチサイズ、ログポリシーに依存する。
6. クォーラムガバナンスは、ガバナンス鍵の安全性と独立性に依存する。

今後の研究では、実行可能な参照実装、BlockMAC 状態機械の形式モデル、正準化のプロパティテスト、再現可能ベンチマーク、SLH-DSA/LMS プロファイル比較、アンカー時刻メタデータのプライバシー分析、およびレジャー内部を開示せずにルール準拠ログ処理を証明するためのゼロ知識検証プロファイルを進めるべきである。

11. 結論

AIKernel Hash-Anchored Trust Layer (HATL) は、高頻度 AI 実行トラックを、すべての内部イベントに公開鍵署名を適用することなく保護するためのトラストレイヤーである。中心的な考え方は、コミット頻度と公開検証頻度を分離することにある。ローカルイベントは高速な対称ラチェットでコミットされ、その後、コンパクトなハッシュベース公開 root としてアンカーされる。

Semantic Context Operating System にとって、この設計は実行性能と監査可能性を橋渡しする。HATL は、ReplayLog を改ざん検知可能な因果記録として保持しつつ、公開検証、Digital Deed ガバナンス、ポリシー駆動の失効を支援する。保証は標準的な暗号仮定と慎重な実装規律に依存するが、AIKernel の将来のトラストインフラに向けた明確な基盤を提供する。

参考文献

参考文献は Part I の References を正とする。